Privacy-Preserving Catalog Intelligence System

Product Requirements Document + Engineering README

Stack: Python · PostgreSQL (Cloud SQL) · Celery · Redis · GCP Cloud Run · scikit-learn
Status: Draft v1.0 — build-from-scratch spec **Owner:** *TBD* **Last updated:** 2026-09-26

---

1. Why this exists (Problem Statement)

Multi-tenant marketplaces and B2B catalog platforms need to enrich, classify, and generate content for product catalogs at scale — but tenants (sellers, brands, retailers) must never see each other's data, even though they share infrastructure, ML models, and background processing pipelines. Building this without a real isolation model tends to produce one of two failure modes:

1. **Silo-per-tenant infrastructure** — safe, but doesn't scale operationally or economically.
2. **Shared tables with app-level filtering** — scales fine, but a single missed `WHERE tenant_id = ...` clause is a data breach.

This system solves it by pushing tenant isolation **into the database engine** via PostgreSQL Row-Level Security (RLS), so isolation is enforced even if application code has a bug. Everything else (Celery, Redis, ML on Cloud Run) is built around that non-negotiable core guarantee.

---

2. Goals / Non-Goals

Goals

- Guarantee cross-tenant data isolation at the database layer, not just the application layer.
- Support asynchronous, horizontally scalable enrichment of large catalogs (auto-classification, description generation, recommendations).
- Keep ML inference isolated per-tenant-context (no tenant's data leaks into another tenant's inference request, cache, or logs).
- Be operable by a small team — no bespoke infra, use managed GCP services wherever sane.
- Be auditable: every data access should be traceable to a tenant + actor.

Non-Goals (v1)

- Not building a multi-region / multi-cloud system.
- Not building real-time (sub-second) inference — this is batch/near-real-time (minutes).
- Not implementing full field-level encryption (that's a v2 concern, noted in Future Work).
- Not a general-purpose feature store — catalog-specific only.

---

3. Users & Use Cases

User	**Use Case**
Tenant admin (seller/brand)	Uploads a catalog (CSV/API), gets back enriched product data: categories, generated descriptions, tags
Platform ops	Monitors pipeline health, reprocesses failed batches, audits access logs
Downstream consumer service	Reads enriched catalog data via API for search/recommendations surfaces
Compliance/security reviewer	Verifies no cross-tenant leakage occurred; pulls audit trail

---

4. High-Level Architecture

```
flowchart LR

  subgraph Client

    A[Tenant Upload / API]

  end

  subgraph API["API Layer (Cloud Run)"]

    B[FastAPI Gateway]
```

```
end

subgraph Queue["Async Layer"]

    C[(Redis - Broker + Result Backend)]

    D[Celery Workers - Cloud Run Jobs / GKE]

end

subgraph DB["Data Layer"]

    E[(Cloud SQL - PostgreSQL w/ RLS)]

end

subgraph ML["ML Layer (Cloud Run, isolated per request context)"]

    F[Classification Service - scikit-learn]

    G[Description/Recommendation Service]

end

A --> B

B -->|sets tenant context via SET LOCAL| E

B -->|enqueue job with tenant_id| C

C --> D

D -->|SET app.tenant_id per task| E

D --> F

D --> G

F -->|writes results scoped to tenant| E

G -->|writes results scoped to tenant| E

B -->|read enriched data, RLS-filtered| E
```

Key architectural principle: every single connection to Postgres — from the API, from a Celery worker, from anywhere — must set the tenant context (`SET LOCAL app.tenant_id = '<uuid>'`) as the *first* statement in its transaction, before any query touches tenant-scoped tables. RLS policies key off this session variable. No code path is allowed to query tenant tables without it.

---

5. Data Model & Row-Level Security Design

5.1 Core tables

```
-- Tenants (not RLS-protected — this table is the source of truth for tenant identity)

CREATE TABLE tenants (

    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),

    name TEXT NOT NULL,

    created_at TIMESTAMPTZ NOT NULL DEFAULT now()

);

-- Catalog items (RLS-protected)

CREATE TABLE catalog_items (

    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),

    tenant_id UUID NOT NULL REFERENCES tenants(id),

    sku TEXT NOT NULL,

    raw_title TEXT,

    raw_description TEXT,

    generated_description TEXT,

    category_path TEXT,
```

```
    classification_confidence NUMERIC(4,3),
    status TEXT NOT NULL DEFAULT 'pending', -- pending | enriched | failed
    created_at TIMESTAMPTZ NOT NULL DEFAULT now(),
    updated_at TIMESTAMPTZ NOT NULL DEFAULT now(),
    UNIQUE (tenant_id, sku)
);
-- Recommendations (RLS-protected)
CREATE TABLE catalog_recommendations (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    tenant_id UUID NOT NULL REFERENCES tenants(id),
    source_item_id UUID NOT NULL REFERENCES catalog_items(id),
    recommended_item_id UUID NOT NULL REFERENCES catalog_items(id),
    score NUMERIC(5,4) NOT NULL,
    created_at TIMESTAMPTZ NOT NULL DEFAULT now()
);
-- Audit log (append-only, RLS-protected but also globally readable by an 'auditor' role)
CREATE TABLE access_audit_log (
    id BIGSERIAL PRIMARY KEY,
    tenant_id UUID NOT NULL,
    actor TEXT NOT NULL,          -- service account / user id
    action TEXT NOT NULL,         -- 'read' | 'write' | 'enrich' | 'export'
```

```
    resource TEXT NOT NULL,       -- table/row reference
    occurred_at TIMESTAMPTZ NOT NULL DEFAULT now()
);
```

5.2 RLS policies

```
-- Enable RLS everywhere it matters. FORCE ensures even the table owner obeys it.
ALTER TABLE catalog_items ENABLE ROW LEVEL SECURITY;
ALTER TABLE catalog_items FORCE ROW LEVEL SECURITY;
ALTER TABLE catalog_recommendations ENABLE ROW LEVEL SECURITY;
ALTER TABLE catalog_recommendations FORCE ROW LEVEL SECURITY;
ALTER TABLE access_audit_log ENABLE ROW LEVEL SECURITY;
ALTER TABLE access_audit_log FORCE ROW LEVEL SECURITY;
-- The tenant context is a per-session GUC. Default to a value that matches nothing
-- if unset, so a forgotten SET LOCAL fails closed, not open.
CREATE POLICY tenant_isolation_select ON catalog_items
    FOR SELECT
    USING (tenant_id = current_setting('app.tenant_id', true)::uuid);
CREATE POLICY tenant_isolation_write ON catalog_items
    FOR ALL
    USING (tenant_id = current_setting('app.tenant_id', true)::uuid)
    WITH CHECK (tenant_id = current_setting('app.tenant_id', true)::uuid);
-- Repeat the same two-policy pattern for catalog_recommendations and access_audit_log.
-- Auditor role can read across all tenants (compliance use case) but never write.
```

```
CREATE ROLE auditor_role;

CREATE POLICY auditor_read_all ON access_audit_log
    FOR SELECT
    TO auditor_role
    USING (true);
```

Fail-closed guarantee: `current_setting('app.tenant_id', true)` returns `NULL` if unset, and `NULL = anything` is `NULL` (falsy) in SQL — so a connection that forgets to set the tenant context sees **zero rows**, never another tenant's rows. This is the single most important property of the whole system; write a test for it (see §9.2) and never let it regress.

5.3 DB roles

Role	Purpose	RLS bypass?
`app_role`	Used by API + Celery workers	No — always scoped
`auditor_role`	Compliance dashboards / audit exports	Read-only, cross-tenant on audit log only
`migration_role` (superuser-ish)	Schema migrations only, never used at runtime	N/A

Never connect to the database at runtime using a superuser or table-owner role — `FORCE ROW LEVEL SECURITY` still doesn't apply to `BYPASSRLS` roles, so this must be enforced by convention + connection string review.

---

6. Application Layer Design

6.1 Tenant context propagation (the one rule that matters most)

Every DB session must go through a single connection helper — never call `psycopg`/`SQLAlchemy` directly elsewhere in the codebase.

```
# db/session.py
```

```
from contextlib import contextmanager
from sqlalchemy import text
from sqlalchemy.orm import Session
from db.engine import SessionLocal
@contextmanager
def tenant_scoped_session(tenant_id: str):
    """
    The ONLY sanctioned way to get a DB session in this codebase.
    Sets the RLS session variable before any other statement runs,
    inside the same transaction, so it can never be bypassed or
    forgotten by a caller.
    """
    session: Session = SessionLocal()
    try:
        session.execute(text("SET LOCAL app.tenant_id = :tid"), {"tid": tenant_id})
        yield session
        session.commit()
    except Exception:
        session.rollback()
        raise
    finally:
```

session.close()

Enforcement, not just convention: add a lint rule / code review checklist item (and ideally a CI grep check) that fails the build if `SessionLocal()` or a raw engine `.connect()` is called anywhere outside `db/session.py`. This is the kind of thing that quietly rots into a vulnerability if left to discipline alone.

6.2 API layer (FastAPI on Cloud Run)

POST /v1/catalogs/{tenant_id}/items -> ingest raw catalog rows

GET /v1/catalogs/{tenant_id}/items/{item_id} -> read one enriched item

GET /v1/catalogs/{tenant_id}/items -> list (paginated)

POST /v1/catalogs/{tenant_id}/reprocess -> re-trigger enrichment for failed items

GET /v1/catalogs/{tenant_id}/recommendations/{item_id}

GET /v1/audit/{tenant_id} -> tenant's own audit trail

GET /v1/admin/audit -> cross-tenant, auditor_role only

- `tenant_id` is extracted from the authenticated JWT/session — **never trust a path or body param for authorization**, only use it as a routing convenience and then cross-check it equals the authenticated principal's tenant before opening the DB session.
- Every request handler wraps its DB work in `tenant_scoped_session(authenticated_tenant_id)`.
- Every mutating request writes one row to `access_audit_log` in the same transaction.

6.3 Async pipeline (Celery + Redis)

Ingestion API

└─ enqueue: enrich_catalog_item(tenant_id, item_id)

└─ Celery worker picks up task

├─ tenant_scoped_session(tenant_id) opens

├─ fetch raw item

├─ call classification service (Cloud Run) -> category_path, confidence

```
    ├─ call description/recommendation service (Cloud Run) -> generated_description
    ├─ write results back inside tenant_scoped_session
    └─ write audit log row
```

Task signature carries `tenant_id` explicitly — it is **never** inferred from global/thread-local state, since Celery workers are shared across tenants and any implicit state is a leak vector.

```
# tasks/enrichment.py

from celery import shared_task

from db.session import tenant_scoped_session

from services.classifier_client import classify

from services.description_client import generate_description

@shared_task(bind=True, max_retries=3, acks_late=True)

def enrich_catalog_item(self, tenant_id: str, item_id: str):

    with tenant_scoped_session(tenant_id) as db:

        item = get_item_or_404(db, item_id)  # query is RLS-scoped automatically

        classification = classify(tenant_id=tenant_id, text=item.raw_title)

        description = generate_description(tenant_id=tenant_id, item=item)

        item.category_path = classification.path

        item.classification_confidence = classification.confidence

        item.generated_description = description

        item.status = "enriched"

        write_audit_log(db, tenant_id, action="enrich", resource=f"catalog_items:{item_id}")
```

Queue design notes:

- Redis used as both Celery broker and result backend; separate logical DB index for each to avoid key collisions.
- Queue routing: separate queues per workload class (`classification`, `description`, `reindex`) so a burst of one tenant's large catalog doesn't starve another tenant's small, latency-sensitive jobs. Use Celery's `task_routes` + priority queues, not per-tenant queues (that doesn't scale past a handful of tenants).
- Idempotency: task keyed by `(tenant_id, item_id, content_hash)` so retries never double-process or double-charge inference cost.
- Backpressure: cap concurrent tasks per tenant using a Redis-based semaphore/rate-limiter so one tenant's huge catalog upload can't monopolize workers.

6.4 ML layer (Cloud Run)

Two independently deployable Cloud Run services:

1. **Classification service** — scikit-learn model (e.g., TF-IDF + linear classifier, or a small transformer if justified later) behind a thin FastAPI wrapper. Stateless. Takes `tenant_id` purely for logging/audit — the model itself is **not** fine-tuned per tenant in v1 (shared global model), so isolation here means "no tenant's inference inputs are cached, logged with content, or batched together with another tenant's request in a way that could leak through timing or batching side effects."
2. **Description/recommendation service** — wraps whatever generation approach is chosen (template-based, small local model, or a hosted LLM API call). Same isolation rules apply.

Isolation rules for the ML layer:

- No request-level caching keyed only on content hash without tenant_id in the key (prevents a cache hit from tenant A silently serving tenant B if content happens to match).
- No batching of requests across tenants into a single model call unless the model call is provably side-effect-free per item (no shared attention context, no cross-item leakage) — flag this explicitly in code comments if batching is added later.
- Structured logs include `tenant_id` and `item_id` but **never** the raw content of another tenant alongside it in the same log line (avoid accidental log aggregation leaks).
- Cloud Run services deployed with **no** persistent local disk state between invocations (default behavior) and env-based min/max instance scaling so cold-start cost is bounded.

---

7. Anti-Gravity: Explicit Build Instructions (Do This, Not That)

This section exists so anyone (or any agent) building this from scratch has a checklist that removes ambiguity. "Anti-gravity" here = the things that will otherwise silently pull the system back down into an insecure or unmaintainable state if you're not deliberate about them.

7.1 Build order (do not skip ahead)

1. **Provision Cloud SQL (Postgres 15+)** with private IP, no public IP exposure. Enable `pgcrypto` extension for `gen_random_uuid()`.
2. **Write and apply the schema migration** (§5.1) using a migration tool (Alembic) — never hand-run DDL against production.
3. **Apply RLS policies** (§5.2) as part of the *same* migration that creates the tables — never ship a table without RLS in the same commit.
4. **Write the RLS fail-closed test first** (§9.2), before writing any application code that reads/writes these tables. If this test doesn't exist and pass, do not proceed.
5. **Build `db/session.py`** (§6.1) as the single sanctioned DB access point. Nothing else touches the engine directly.
6. **Build the FastAPI ingestion + read endpoints**, each wrapped in `tenant_scoped_session`.
7. **Stand up Redis + Celery**, wire the `enrich_catalog_item` task, run it against a stub classifier/description client first (return fake data) to validate the async plumbing end-to-end before touching real ML.
8. **Build and deploy the classification service** to Cloud Run, swap the stub client for the real HTTP client.
9. **Build and deploy the description/recommendation service**, same swap.
10. **Add the audit log writes** to every mutating path (should already be threaded through if you followed steps 5–9 correctly, since the session helper can enforce it).
11. **Load test with synthetic multi-tenant data** — specifically include an adversarial test tenant that tries to reference another tenant's `item_id` in every endpoint and confirm it always 404s, never 403s (don't leak existence via status code).
12. **Security review pass** against the checklist in §7.2 before any production data touches the system.

7.2 Non-negotiable rules ("no shit coding" checklist)

- ☐ No raw SQL string interpolation, anywhere. Parameterized queries only.
- ☐ No DB session created outside `db/session.py`.
- ☐ No table containing tenant data ships without `ENABLE ROW LEVEL SECURITY` **and** `FORCE ROW LEVEL SECURITY` in the same migration.
- ☐ No Celery task infers `tenant_id` from anything other than an explicit function argument.

- [ ] No endpoint trusts a `tenant_id` from the URL/body without cross-checking it against the authenticated principal.
- [ ] No log statement includes full raw catalog content from more than one tenant in the same line/structured record.
- [ ] No secrets (DB creds, API keys) in code or env files committed to git — use GCP Secret Manager, injected at runtime.
- [ ] Every migration is reversible (`downgrade()` implemented, not just `upgrade()`).
- [ ] Every new mutating endpoint has a corresponding audit log write, enforced by a shared decorator/helper, not copy-pasted per-endpoint.
- [ ] CI blocks merge if the RLS fail-closed test (§9.2) doesn't exist or fails.
- [ ] No `BYPASSRLS` or table-owner role used by any runtime service account.

7.3 Definition of Done (per feature)

A feature (e.g., "auto-classification") is done when:

- Unit tests pass, including a negative test proving cross-tenant access is impossible.
- The feature's DB access goes exclusively through `tenant_scoped_session`.
- Audit log entries are produced and verified in a test.
- Load-tested with at least 2 concurrent tenants processing simultaneously with no cross-contamination.
- Structured logs reviewed for accidental data leakage.
- Runbook entry added (§10) for how to detect and recover from a failure in this feature.

---

8. Tech Stack Summary

Layer	**Choice**	**Why**
API	FastAPI	Async-native, typed, plays well with SQLAlchemy 2.0 async sessions
DB	PostgreSQL on Cloud SQL	Native RLS support, mature, GCP-managed
ORM/migrations	SQLAlchemy 2.0 + Alembic	Explicit control over raw SQL when needed (RLS policies aren't ORM-native)
Queue/broker	Redis	Simple, fast, doubles as Celery result backend

Layer	Choice	Why
Task runner	Celery	Mature retry/backoff semantics, good observability tooling
ML	scikit-learn	Lightweight, explainable, cheap to serve for classification
ML/API hosting	Cloud Run	Scale-to-zero, per-service isolation, simple deploys
Secrets	GCP Secret Manager	No secrets in code/env files
Observability	Cloud Logging + Cloud Monitoring, structured JSON logs	Native GCP integration

---

9. Testing Strategy

9.1 Test pyramid

- **Unit tests:** business logic, serializers, task logic (with DB mocked).
- **Integration tests:** real Postgres (via test container) exercising actual RLS policies — this is the most important tier for this system.
- **Contract tests:** classification/description Cloud Run services tested independently with fixed input/output contracts.
- **End-to-end tests:** ingest → enrich → read, across 2+ synthetic tenants.

9.2 The RLS fail-closed test (write this before anything else)

```
def test_rls_blocks_cross_tenant_access(db_engine):

    tenant_a, tenant_b = create_test_tenants(db_engine)

    item = insert_catalog_item(db_engine, tenant_id=tenant_a.id, sku="SKU-1")

    with tenant_scoped_session(tenant_b.id) as db:

        result = db.execute(
```

```
            select(CatalogItem).where(CatalogItem.id == item.id)
        ).first()
        assert result is None  # tenant B must see zero rows, not an error, not tenant A's row

def test_rls_fails_closed_when_context_unset(db_engine):
    # Simulate a bug where SET LOCAL was never called
    with SessionLocal() as db:  # bypasses tenant_scoped_session on purpose, for this test only
        result = db.execute(select(CatalogItem)).all()
        assert result == []  # must be empty, never "all rows" (that would mean RLS isn't forced)
```

Run both tests in CI on every PR that touches the schema or `db/session.py`.

---

10. Operations & Runbook (starter set)

Scenario	**Detection**	**Response**
Celery queue backlog growing	Cloud Monitoring alert on queue depth	Scale worker instance count; check for a stuck/looping task
Classification service latency spike	Cloud Run request latency alert	Check model load time / cold starts; consider min-instances > 0
Suspected cross-tenant leak	Audit log anomaly (tenant A's actor touching tenant B's resource string)	Immediately revoke the offending service account, freeze writes, run RLS test suite against prod schema
Failed enrichment batch	`status = 'failed'` rows accumulating	`POST /v1/catalogs/{tenant_id}/reprocess`, investigate root cause before mass-retry

---

11. Future Work (explicitly out of scope for v1)

- Per-tenant fine-tuned models (would require rethinking the "shared model" isolation assumptions in §6.4).
- Field-level encryption at rest for `raw_description` / PII-adjacent fields.
- Multi-region active-active Cloud SQL (currently single-region with standard HA).
- Fine-grained per-column RLS (currently row-level only).

---

12. Repo Structure (suggested)

.

```
├── api/                # FastAPI app, routers, request/response schemas
├── db/
│   ├── session.py      # the ONE sanctioned DB access point
│   ├── models.py
│   └── migrations/     # Alembic
├── tasks/              # Celery task definitions
├── services/
│   ├── classifier_client.py
│   └── description_client.py
├── ml_services/
│   ├── classification/   # deployed independently to Cloud Run
│   └── description/      # deployed independently to Cloud Run
├── tests/
│   ├── unit/
```

```
│   ├── integration/      # real-Postgres RLS tests live here
│   └── e2e/
├── infra/                # Terraform for Cloud SQL, Cloud Run, Redis (Memorystore)
└── README.md
```

This document is meant to be a living spec — update §7.2 and §9.2 first if any architectural assumption changes, since everything else depends on them holding true.